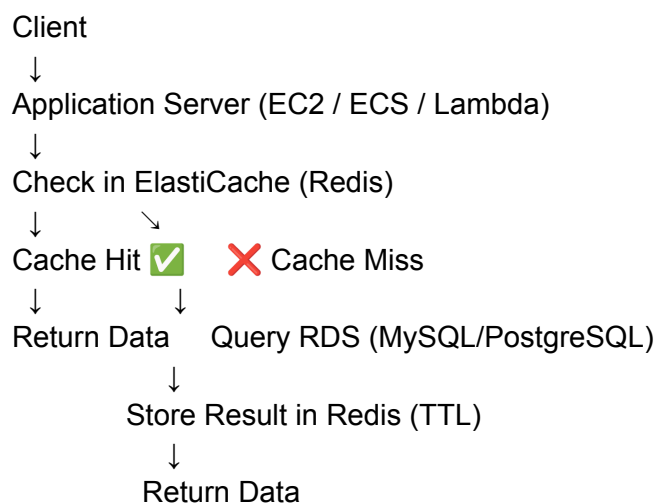

Using Amazon ElastiCache with Amazon RDS

Goal

Improve performance of your database-driven application (e.g., e-commerce, blog, analytics app) by:

- Reducing **latency**
 - Decreasing **RDS load**
 - Improving **scalability**
-

Architecture Overview



Use Case Example

 *You're building an e-commerce app.*

Each time a user views a product, your app:

1. Checks Redis (ElastiCache) for product info
 2. If not found, queries RDS (MySQL)
 3. Stores the result in Redis with an expiry (TTL)
 4. Returns the data to the user
-

Step-by-Step Setup

Step 1: Create RDS Instance (MySQL/PostgreSQL)

1. Go to **Amazon RDS** → **Databases** → **Create database**
 2. Choose:
 - **Engine:** MySQL / PostgreSQL
 - **Instance type:** `db.t3.micro` (free tier)
 - **Storage:** General Purpose SSD
 - Enable Public Access if using from EC2 (or keep private if inside VPC)
 3. Set username/password
 4. Note the **endpoint** after creation
-

Step 2: Create ElastiCache Redis Cluster

1. Go to **Amazon ElastiCache** → **Redis** → **Create**
2. Cluster Mode: **Disabled** (single node for now)
3. Node type: `cache.t3.micro`
4. VPC: Same as your RDS + EC2
5. Enable:
 - In-transit encryption
 - Redis AUTH (save token securely)

Wait until **Status: Available**

✓ **Step 3: Launch EC2 Instance**

1. Launch an EC2 instance in the **same VPC + subnet**
 2. Attach a **Security Group** that allows:
 - Port **3306** (for RDS)
 - Port **6379** (for Redis)
 3. SSH into EC2
-

✓ **Step 4: Install MySQL Client + Redis Tools**

```
sudo apt update
sudo apt install mysql-client redis-tools -y
```

Test connection:

```
mysql -h <rds-endpoint> -u admin -p
redis-cli -h <redis-endpoint> -a <auth-token>
```

✅ Step 5: Application Logic (Sample Pseudo-Code)

Python Example

```
import redis
import mysql.connector
import json

# Connect to Redis
r = redis.Redis(host='your-redis-endpoint', port=6379, password='your-token',
decode_responses=True)

# Connect to RDS
db = mysql.connector.connect(
    host="your-rds-endpoint",
    user="admin",
    password="yourpass",
    database="products"
)

cursor = db.cursor(dictionary=True)

product_id = 101
cache_key = f"product:{product_id}"

# 1. Check cache
cached = r.get(cache_key)
if cached:
    print("From Cache:", json.loads(cached))
else:
    # 2. Query RDS
    cursor.execute("SELECT * FROM items WHERE id = %s", (product_id,))
    result = cursor.fetchone()

    # 3. Cache result
    r.setex(cache_key, 300, json.dumps(result)) # TTL 5 minutes

    print("From RDS:", result)
```

Component

Tip

Redis	Use AUTH tokens, encryption, limit SG access
RDS	Disable public access, use IAM/SSL
EC2	Use IAM role for app logs/metrics
Network	Keep all components in private subnets with proper routing



Performance Benefits

With ElastiCache



Fast responses
(microseconds)



Lower RDS CPU load



Fewer DB connections



Cost savings at scale

Without ElastiCache



Slower queries (milliseconds)



Higher DB load



Connection scaling required



RDS scale-up costs more



Monitoring

- Use **CloudWatch** to monitor:
 - RDS query counts, CPU, connections
 - Redis: Memory usage, cache hits/misses, TTLs
 - Use **ElastiCache Metrics Dashboard**
-



Student Lab Idea



"Build a cache layer for your database-driven API"

1. Deploy a Flask or Node.js app on EC2
2. Connect to RDS to fetch data
3. Use Redis (ElastiCache) to cache results
4. Simulate load using `ab` or `curl` loop
5. Observe performance difference in response time & CPU load

Summary

Component	Purpose
RDS	Persistent, durable relational data store
ElastiCache	Fast in-memory caching layer
EC2 / Lambda	App logic
Benefit	Lower latency, improved scalability, reduced DB cost
